Day 17 · Chatbots & Language AI — Differentiated Handout
Advanced AI & CS · Grades 5–6 · Mr. Hsiao "Hue" Weng · Spring 2026
Lesson Question: *What does it mean for a machine to understand language — and why is that question harder than it sounds?*

Learning Objectives

By the end of today, you will be able to:

- Explain **tokenization** and how it converts sentences into data a computer can process
- Build and trace a **bag-of-words scorer** that classifies intent using weighted keywords
- Identify the **bugs** in TinyBot v1 and articulate *why* they fail
- Describe what a **Finite State Machine (FSM)** adds to a stateless chatbot
- Compare TinyBot to GPT-4 across at least four dimensions using correct CS vocabulary

Key Vocabulary

Term	Plain-English Definition
Token	The smallest unit of text a program processes (word, subword, or character)
Tokenization	Splitting a sentence into tokens before any analysis
Bag-of-Words	Representing text as a count/weight of words, ignoring word order
Intent	The purpose or goal behind a message ("greeting", "help", "negative")
Deterministic	Same input → same output, every time; no randomness
Finite State Machine (FSM)	A model with a fixed set of states and rules for transitioning between them
Big-O Notation	A way to describe how an algorithm's speed scales with input size
Hallucination	When an AI generates a confident but false statement
Grounding	Connecting a word's meaning to real-world sensory experience
RLHF	Reinforcement Learning from Human Feedback — how GPT is made safer

Warm-Up: Think Like a Computer (5 min)

Work through these puzzles in your notebook before class discussion.

Puzzle 1 — Ambiguity

The sentence *"I saw the man with the telescope"* has **two** different meanings. Draw both interpretations as a tree diagram. A computer sees only one string of characters — which meaning would *you* choose, and how?

→ This is the **syntactic ambiguity** problem at the heart of NLP.

Puzzle 2 — Negation

Write a **one-line Python expression** (no import) that returns True only when "sad" appears but "not" does NOT.

```
python
# Fill in the blank:
result = _____________________________________________
```

Why do real NLP systems need more than just this check?

Puzzle 3 — Complexity

TinyBot has 6 intent rules. If each rule adds one elif check taking constant time, what is the **Big-O time complexity** of intent classification?

O(_____) where the variables stand for: _____

→ Could this bottleneck at 10 million messages/day?

Core Concept 1 — Tokenization

Before any AI reads a sentence, it **tokenizes** it — splits it into small units called *tokens*.

```python
def tokenize(sentence):
    import re
    clean = re.sub(r'[^a-z\s]', '', sentence.lower())
    return clean.split()

tokens = tokenize("I'm NOT sad!")
# → ['im', 'not', 'sad']
```

Investigation: The regex removes apostrophes, turning "I'm" into "im". Are "im" and "I'm" the same token *semantically*? What meaning is lost? How would you fix this?

GPT-4 scale: Trained on ~13 trillion tokens. Each ≈ ¾ of a word. That's roughly 10 trillion words — every Wikipedia article ~50,000 times over.

Core Concept 2 — Bag-of-Words Intent Scoring

Instead of a brittle if "sad" in msg check, assign **numerical weights** to keywords and compute a score. The highest score wins.

Python

```
INTENTS = {
    'greeting': {'hello': 3, 'hi': 3, 'hey': 2},
    'negative': {'sad': 3, 'tired': 2, 'upset': 3, 'depressed': 4},
    'positive': {'happy': 3, 'great': 2, 'excited': 3, 'awesome': 3},
}

def classify(msg):
    tokens = msg.lower().split()
    scores = {intent: 0 for intent in INTENTS}
    for token in tokens:
        for intent, weights in INTENTS.items():
            scores[intent] += weights.get(token, 0)
    best = max(scores, key=scores.get)
    return best if scores[best] > 0 else 'unknown'
```

Trace this input: "I feel a bit sad and tired"

Intent	Score	Matched Keywords
greeting		
negative		
positive		

Winner →

Still broken: "I'm not sad at all, I feel great!" scores negative=3, positive=2 → wrong winner. What linguistic feature would fix this? *(Hint: look 1 token to the left of each keyword.)*

Core Concept 3 — TinyBot v1 (Fully Annotated)

python

```
# TinyBot v1 — DETERMINISTIC: same input → same output. Always.
print('Welcome to TinyBot! Type "bye" to quit.')

while True:                                # infinite event loop
    msg = input('You: ').lower().strip() # normalise + clean

    # INTENT DISPATCH: O(k) where k = number of elif branches
    if msg == 'bye':
        print('Bot: Goodbye!')
        break
    elif 'hello' in msg or 'hi' in msg:  # BUG: 'hi' matches 'this', 'white'
        print('Bot: Hi there!')
    elif 'name' in msg:
        print('Bot: I am TinyBot.')
    elif 'sad' in msg or 'tired' in msg: # BUG: 'not sad' still triggers this
        print('Bot: Breaks and friends can really help.')
    elif 'happy' in msg or 'excited' in msg:
        print('Bot: That is awesome news!')
    else:
        print('Bot: I do not understand yet.')
```

Bug Hunt — label each bug type:

Bug	What input triggers it? Why it fails
'hi' in msg substring match	
First-match bias	
No memory between turns	

Core Concept 4 — Data-Driven v2 (Dict Architecture)

python

```
RULES = {
    'greeting': ['hi', 'hello', 'hey'],
    'farewell': ['bye', 'goodbye'],
    'negative': ['sad', 'tired', 'upset', 'unhappy'],
    'positive': ['happy', 'excited', 'great', 'awesome'],
}
REPLIES = {
```

```
    'greeting': 'Hi there!',
    'farewell': 'Goodbye!',
    'negative': 'Sorry to hear that. Want help?',
    'positive': 'Awesome!',
    'unknown':  'I do not understand yet.',
}

def get_intent(msg):
    tokens = set(msg.lower().split())   # O(n) once
    for intent, kws in RULES.items():
        if tokens & set(kws):           # set intersection O(min)
            return intent
    return 'unknown'
```

Why sets beat lists: List intersection = O(n×m). Set intersection = O(min(n,m)). At 1,000 keywords, sets are ~50× faster.

Core Concept 5 — Finite State Machine

TinyBot v1 is **stateless** — it forgets everything between turns. An FSM stores a state variable that persists.

text

```
START → (greeting) → ASK_NAME → (name given) → KNOW_NAME → (bye) → END
```

python

```
state = 'start'
name  = None

while True:
    msg = input('You: ').lower().strip()

    if state == 'start':
        if any(w in msg.split() for w in ['hi', 'hello', 'hey']):
            print("Bot: Hi! What's your name?")
            state = 'ask_name'          # ← transition
        else:
            print('Bot: Say hello first!')

    elif state == 'ask_name':
        name = msg.title()
        print(f'Bot: Nice to meet you, {name}!')
        state = 'know_name'

    elif state == 'know_name':
        if msg == 'bye':
            print(f'Bot: Goodbye, {name}!'); break
        else:
            print(f'Bot: What else, {name}?')
```

Formal notation: FSM = (Q, Σ, δ, q_0, F) — states, inputs, transition function, start state, accepting states.

Tiered Coding Challenges (15 min — Choose Your Level)

Level 1 — Refactor *(Approaching)*

Convert TinyBot v1's if/elif chain to the **data-driven dict structure** from Core Concept 4. Every reply must still work. Adding a new intent should require **only a new dict entry**, no new code.
Success metric: Add a 'weather' intent in under 60 seconds.
Python

```
# Starter — fill in the blanks:
RULES  = { 'greeting': ['hi', 'hello'], _____ }
REPLIES = { 'greeting': 'Hi there!', _____ }

def get_intent(msg):
    tokens = _____
    for intent, kws in RULES.items():
        if _____:
            return intent
    return 'unknown'
```

Level 2 — Weighted Scorer *(On Grade Level)*

Implement the **bag-of-words classifier** from Core Concept 2, then add the **negation window fix**.
Test on:

- "I'm not sad at all" → expected: unknown or positive
- "I feel a little excited and kind of sad" → which intent wins?

Python
```
# Negation window extension:
for i, token in enumerate(tokens):
    neg  = (i > 0 and tokens[i-1] in ['not', "n't", 'never'])
    sign = -1 if neg else 1
    scores[intent] += sign * weights.get(token, 0)
```

Explain in 1 sentence: Why does dict.get(key, 0) matter here?

Level 3 — FSM Dialogue *(Extended)*

Build a **4-state FSM**: start → ask_name → ask_mood → chat.

- In chat state, use your weighted scorer from Level 2
- Personalize every reply using the stored name variable
- **Extension:** if mood score is negative, transition to a support state that always suggests talking to a teacher

Draw the FSM transition diagram using (Q, Σ, δ, q_0, F) notation before coding.

Real-World Connections

Your Code Today	Real System	Scale
tokenize() word splitter	GPT-4 BPE tokenizer	100K vocab tokens
classify() scorer	Gmail spam filter	~15B emails/day
get_intent() dict lookup	Alexa NLU pipeline	Millions of queries/hour
FSM state variable	Game AI behavior trees	1,200+ behavior nodes
Bug testing	Adversarial ML red-teaming	Dedicated safety teams

Homework — Due Next Class

Required — Coding:
Complete **Level 1 or Level 2** (your choice). Add a test() function that runs **8 test messages automatically** and prints PASS/FAIL for each. Include at least 2 tests designed to *break* your bot.
Python
```
def test():
    cases = [
        ("hi there",    "greeting"),
        ("I'm not sad", "unknown"),   # should fail TinyBot v1
        # add 6 more...
    ]
    for msg, expected in cases:
        result = get_intent(msg)
        status = "✓ PASS" if result == expected else "✗ FAIL"
        print(f"{status} | {msg!r} → {result}")
```

Required — Analysis (5–7 sentences):
Use the **Chinese Room argument** and the **grounding problem** to answer: Does TinyBot *understand* language? Does GPT-4? What would a machine need to do to genuinely understand?

Extension — Level 3 + FSM:
Complete Level 3. Formally define its FSM: list every state Q, every transition trigger in Σ, the full transition function δ, the start state q_0, and all accepting states F.

Research Extension:
Look up **Naive Bayes classification** — the algorithm behind early spam filters. Write pseudocode for training it on 10 labelled chatbot messages. How does it differ from your weighted scorer?
Submit to: hue.weng.88@gmail.com · Subject: Day 17 HW · [Your Name] · Level [1/2/3]

Looking Ahead — Day 18

Next lesson: train a **real Naive Bayes classifier** on labelled data and measure **precision**, **recall**, and **F1 score**. Your test() function from tonight's homework is the starting point.
Return to your warm-up puzzles — could you answer Puzzle 1 differently now? What did building TinyBot teach you that reading about it never could?